

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4) Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A APPLICATION BASED QUESTIONS

Code of Conduct:

*I N. Tharun Sai / (192472382) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

N. Tharun Sai

Signature of the student:

ANNEXURE A

1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.
2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.
3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.

4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.
5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.

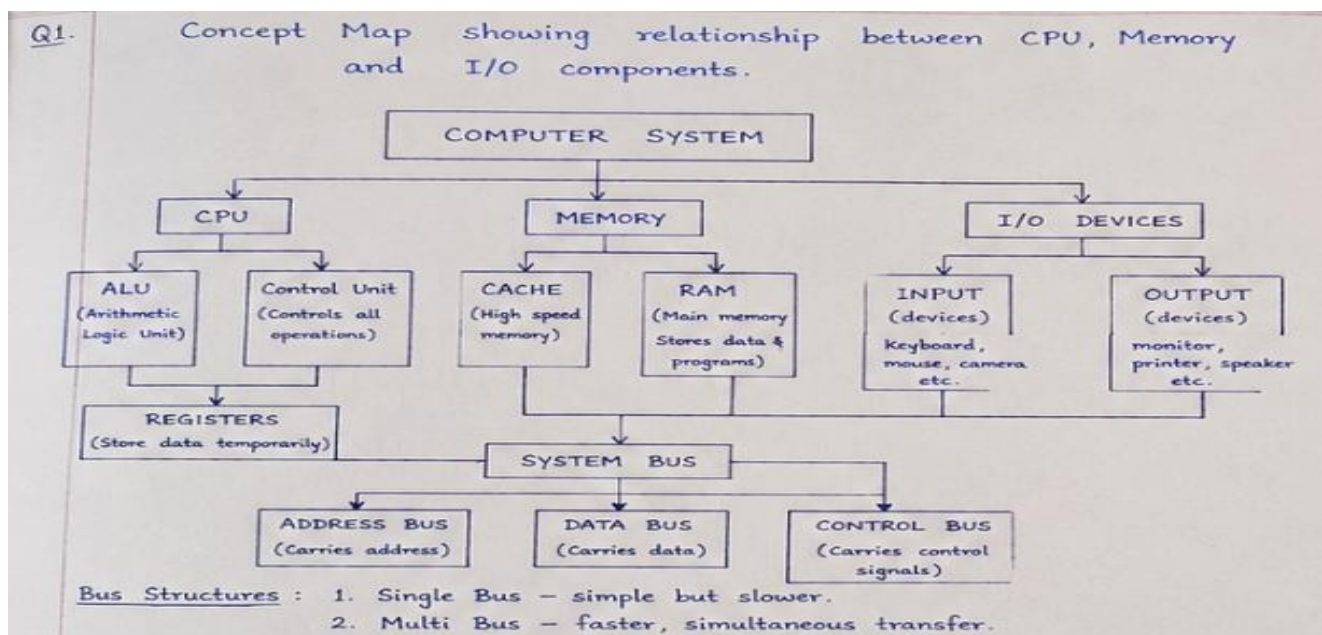
Q1. Concept Map Showing the Relationship Between CPU, Memory, and I/O Devices

Introduction

A computer system consists of three major components: **CPU, Memory, and I/O devices**. The CPU processes instructions, memory stores data and programs, and I/O devices allow communication with external devices. These components are connected through the **system bus**, which includes the address bus, data bus, and control bus. Efficient communication between these units improves overall system performance.

Applications

- Smart city surveillance systems
- Banking systems
- Personal computers
- Mobile devices
- Industrial automation



Explanation

- **CPU** executes instructions using the ALU and Control Unit.
- **Cache Memory** stores frequently used data for faster access.
- **RAM** stores currently running programs.
- **I/O Devices** transfer data between the computer and external devices.
- **Single Bus** is simple but can become a bottleneck.
- **Multi-Bus** allows multiple transfers simultaneously, improving performance.

Sample Program (8085)

```
MVI A,25H  
MVI B,15H  
ADD B  
STA 2500H  
HLT
```

Conclusion

Efficient interaction between the CPU, memory, and I/O devices through advanced bus structures improves execution speed and system performance. The CPU, memory, and I/O devices are the three fundamental components of a computer system. Their coordination through the system bus ensures smooth execution of programs and efficient data transfer. Using advanced bus architectures such as **multi-bus** and **hierarchical bus structures**, along with **cache memory**, reduces communication delays and increases system performance. Therefore, an efficient computer architecture provides faster processing, better resource utilization, and improved overall system reliability.

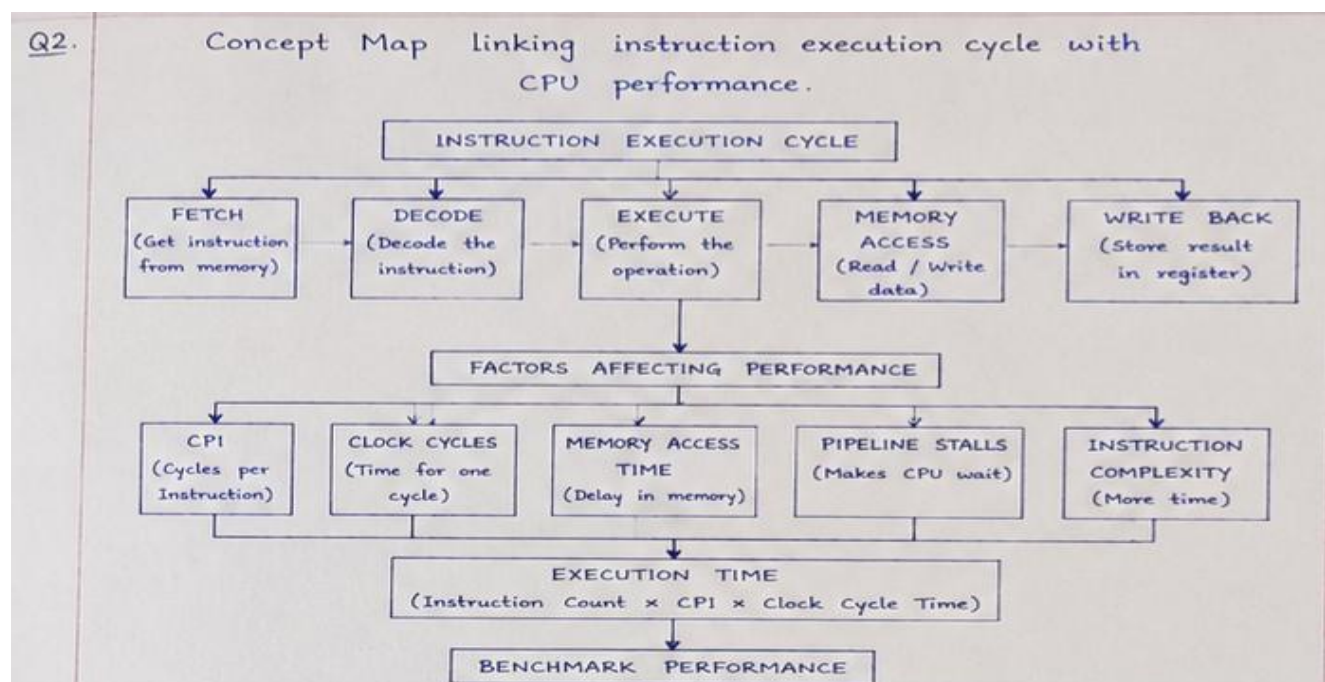
Q2. Concept Map Linking the Instruction Execution Cycle with CPU Performance

Introduction

A real-time stock trading system processes millions of instructions every second to buy and sell shares quickly. Speed is very important because even a small delay can affect profits. The processor follows the fetch–decode–execute cycle to execute every instruction. During high workloads, frequent memory access and branch instructions increase the Cycles Per Instruction (CPI), making the processor slower. Using modern processor techniques can reduce execution time and improve system performance.

Applications

- Stock trading systems
- Online banking
- Gaming processors
- Artificial Intelligence
- Cloud computing



Explanation

- **Fetch:** Reads the instruction from memory.
- **Decode:** Determines the operation.
- **Execute:** Performs the instruction.
- **Memory Access:** Reads or writes data.
- **Write Back:** Stores the result.

- Lower **CPI** and fewer pipeline stalls improve performance.

Sample Program (8085)

MVI A,05H

MVI B,03H

ADD B

MOV C,A

HLT

Conclusion

The fetch–decode–execute cycle affects processor speed. High CPI reduces performance, but techniques like pipelining, cache memory, branch prediction, and multithreading improve execution speed and make stock trading systems faster and more reliable. The instruction execution cycle is the foundation of CPU operation, and every instruction passes through the stages of **fetch, decode, execute, memory access, and write back**. CPU performance depends on factors such as **CPI, clock cycles, memory access time, and pipeline efficiency**. Modern architectural techniques like **instruction pipelining, cache memory, branch prediction, and superscalar execution** significantly reduce execution time and improve throughput. Benchmarking helps evaluate processor performance and compare different systems for real-world applications.

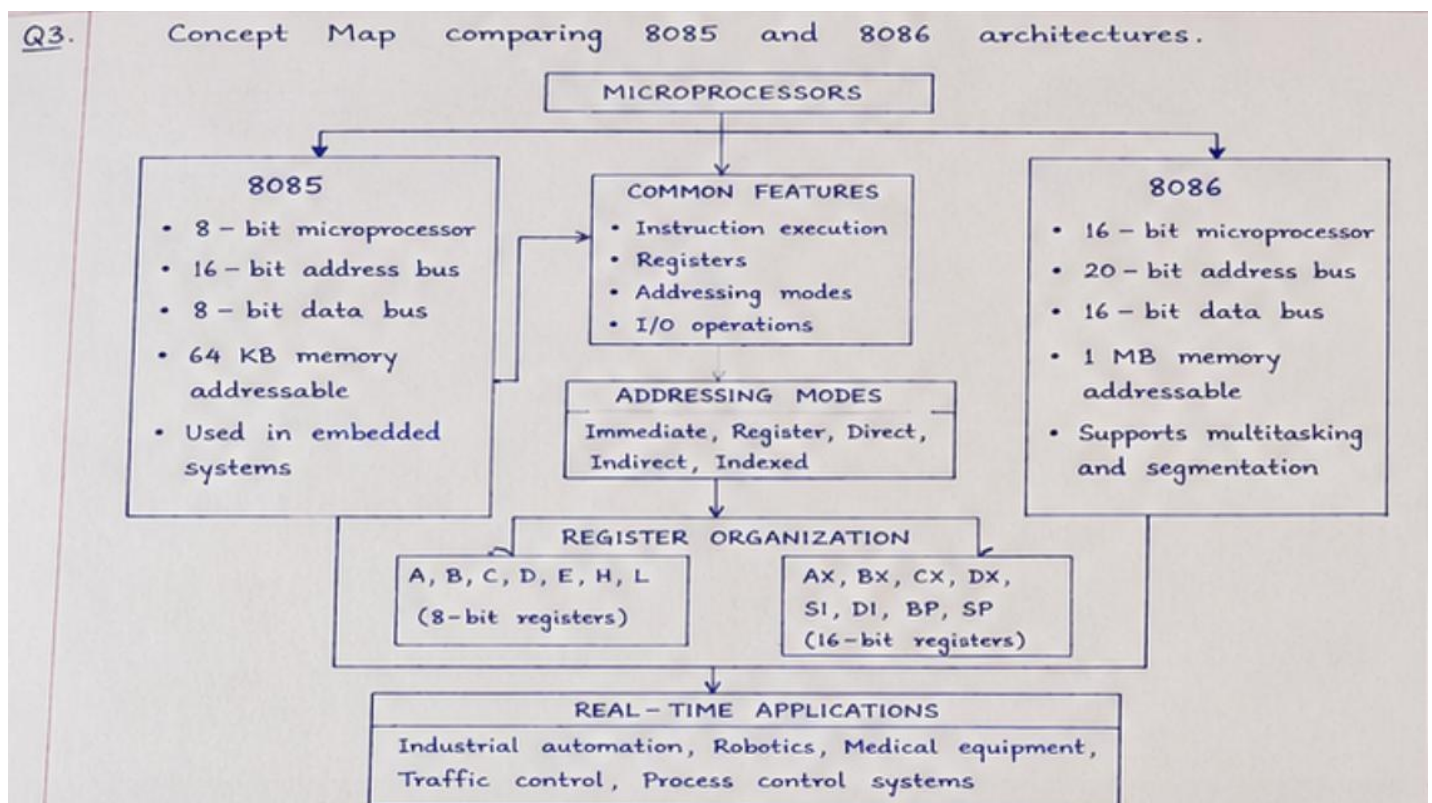
Q3. Concept Map Comparing 8085 and 8086 Microprocessors

Introduction

The 8085 is an 8-bit microprocessor, while the 8086 is a 16-bit microprocessor with better speed, memory organization, and multitasking capability. Both support different instruction sets and addressing modes for efficient program execution.

Applications

- Embedded systems
- Industrial automation
- Robotics
- Medical equipment
- Traffic control systems
- Process control



Explanation

8085

- 8-bit processor
- 16-bit address bus

- 64 KB memory
- Simpler instruction set

8086

- 16-bit processor
- 20-bit address bus
- 1 MB memory
- Segmented memory (CS, DS, SS, ES)
- Faster execution and multitasking

Sample Program (8085)

MVI A,10H

ADI 20H

STA 2500H

HLT

Conclusion

The correct use of 8085 addressing modes ensures accurate data access and reliable temperature control. Proper programming, correct memory addressing, and validation of sensor readings improve the performance and reliability of embedded systems used in industries. The **8085** and **8086** microprocessors play an important role in computer architecture and embedded system design. While the **8085** is suitable for simple and low-cost applications, the **8086** provides higher processing speed, larger memory support, and memory segmentation, making it more efficient for multitasking and advanced applications. Understanding their instruction sets, register organization, and addressing modes helps in designing reliable and efficient programs for industrial automation, robotics, medical devices, and other real-time systems.

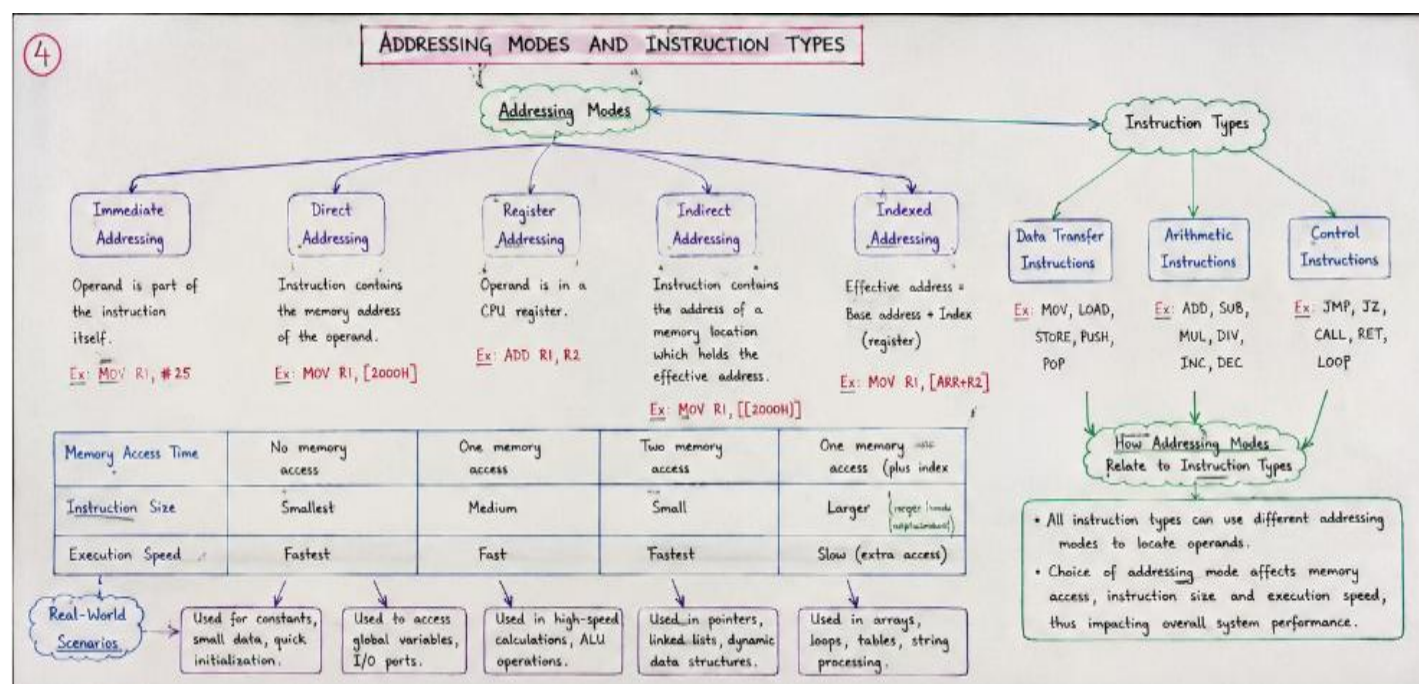
4. Addressing Modes and Instruction Types

Introduction

Addressing modes are techniques used by the CPU to locate the data (operand) required for executing an instruction. They determine how the processor accesses information from registers or memory. Different instruction types, such as data transfer, arithmetic, and control instructions, use different addressing modes to improve performance. Choosing the appropriate addressing mode helps reduce memory access time, optimize instruction size, and increase execution speed.

Explanation

1. Addressing modes specify how the CPU finds the operand for an instruction.
2. Immediate addressing stores the operand inside the instruction, giving the fastest execution.
3. Direct addressing accesses data from a fixed memory location.
4. Register addressing uses CPU registers, reducing memory access time.
5. Indirect addressing accesses data through another address or register.



Applications

- Arithmetic operations
- Data transfer instructions
- Array processing
- Pointer handling
- Loop execution
- Memory access

Conclusion

Addressing modes improve processor efficiency by providing multiple ways to access data. Selecting the appropriate addressing mode reduces execution time, optimizes memory usage, and improves overall system performance. Different addressing modes are suitable for different programming and hardware requirements. Addressing modes play a vital role in determining how the CPU accesses data and executes instructions. Each addressing mode has its own advantages depending on the type of operation being performed. Immediate and register addressing provide faster execution, while direct, indirect, and indexed addressing offer greater flexibility for accessing memory and complex data structures. Selecting the appropriate addressing mode improves processor efficiency, reduces memory access time, optimizes instruction size, and enhances overall system performance. Understanding these addressing modes is essential for designing efficient programs and computer architectures.

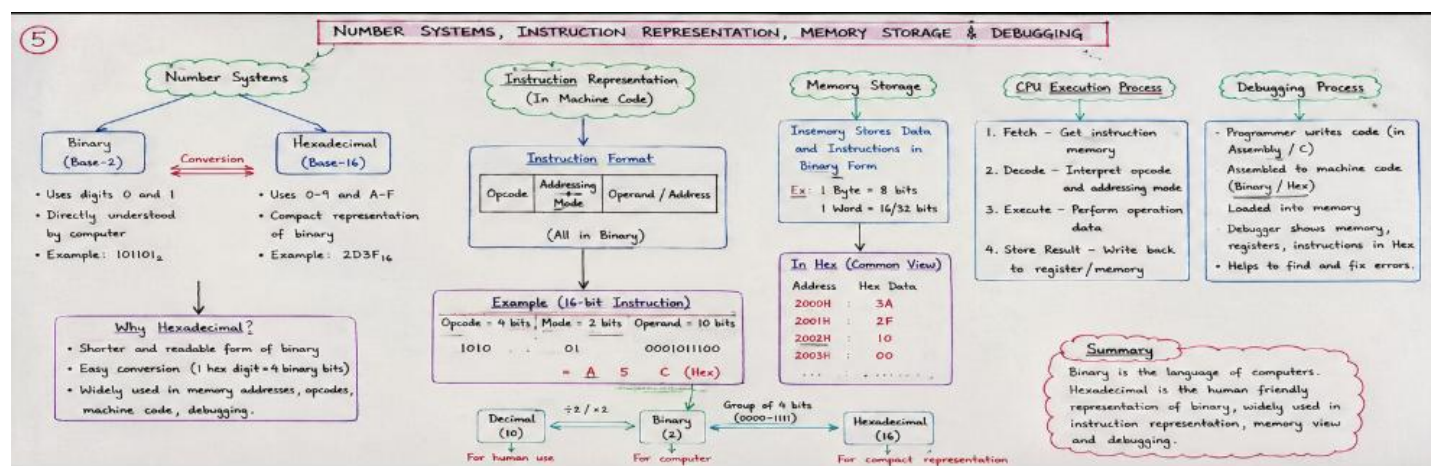
5. Number Systems, Instruction Representation, Memory Storage and Debugging

Introduction

Number systems are the foundation of computer architecture because computers process and store all data in binary form. Hexadecimal is commonly used as a compact and readable representation of binary data. Machine instructions are stored in memory in binary format and are interpreted by the CPU during execution. Understanding binary, hexadecimal, instruction representation, memory storage, and debugging is essential for learning how computer systems execute programs efficiently.

Applications

- Used in assembly language and machine language programming.
- Used in memory address representation.
- Used in embedded systems and microcontroller programming.
- Used in operating system and compiler development.
- Used in debugging tools, disassemblers, and memory analysers.
-



Conclusion

Binary and hexadecimal number systems are fundamental to the operation of digital computers. Binary is used internally by the CPU to represent instructions and data, while hexadecimal provides a compact and human-readable way to display the same information. The CPU fetches, decodes, and executes binary instructions stored in memory, and debugging tools use hexadecimal values to simplify program analysis and error detection. A strong understanding of number systems, instruction representation, memory organization, and debugging techniques helps programmers and engineers develop efficient software, analyze system behavior, and troubleshoot hardware and software issues effectively.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand